

对称密码算法的软件实现与优化

AES / SM4 / GIFT / TWINE 与 CTR / GCM / XTS

课程项目实施与实验报告

2026 年 7 月

摘要

本项目从统一的 C11 标量基线出发，实现 AES-128/192/256、SM4-128、GIFT-64 (128-bit key) 和 TWINE-80/128，并系统比较 T-table、shuffle/bitslice/fixslice 与新指令集。工作模式覆盖全部算法的 CTR，以及 AES/SM4 的 GCM、XTS；GCM 使用 ARM PMULL 加速 GHASH，XTS 同时支持 IEEE 1619 与 GB/T 17964-2021 的 tweak 乘法约定及 ciphertext stealing。Apple M2 Pro 与 Intel Core i9-13900H 上分别保存 3 次预热、15 次正式测量的全部样本。x86-64 原生验证执行 AES-NI、VAES、SSSE3、GFNI、PCLMUL 和 VPCLMUL，包含 2755 项通用检查、830 项专用后端检查与 20160 项 OpenSSL 差分。Intel VSM4 后端已完成适配和静态指令检查，但实验使用的 Core i9-13900H 不支持该指令集，因此没有 VSM4 原生性能数据。

关键词：对称密码；AES；SM4；GIFT；TWINE；T-table；NEON TBL；PMULL；GFNI；CTR；GCM；XTS

目录

1	实现顺序与交付边界	3
2	微架构基础与优化模型	3
2.1	吞吐率不是指令条数	3
2.2	端序与未定义行为	4
3	四种分组密码的基础实现	4
3.1	AES	4
3.2	SM4	4
3.3	GIFT-64 与 TWINE	5
4	统一接口与运行时分派	5
5	优化技术	6
5.1	T-table: 空间换指令	6
5.2	Shuffle、bitslice 与 fixslice	8
5.3	新指令集	9
6	CTR、GCM 与 XTS	10
6.1	CTR	10
6.2	GCM	11
6.3	XTS	13
7	测试与工程验证	14
8	实验方法	15
9	结果与分析	16
9.1	ARM64 原生执行结果	16
9.2	x86-64 原生执行结果	18
9.3	ARM64 与 x86-64 对照分析	22
10	安全、性能与平台权衡	23
11	结论	24

1 实现顺序与交付边界

项目按照“先建立正确性基线，再逐层替换热点”的顺序实现：

1. 先完成 AES、SM4、GIFT 与 TWINE 的单块加/解密和密钥扩展；
2. 用官方或论文向量固定参考输出，形成所有优化后端的比较基线；
3. 为 AES/SM4 加入 T-table，对比表尺寸、缓存占用与吞吐率；
4. 加入 NEON TBL、x86 PSHUFB 和 GIFT bitslice，并把单块接口扩展为四路/八路批量内核；
5. 接入 AES、GFNI、PMULL/PCLMUL 等指令，通过运行时特征检测选择可执行后端；
6. 最后让 CTR、GCM 与 XTS 调用批量分组接口，并补充边界测试、OpenSSL 差分、sanitizer 和六档长度基准。

后续章节按同一顺序展开具体实现：先说明如何把 AES/SM4 的非线性层与线性层合并成 T-table，再说明 shuffle/bitslice 与密码指令怎样替换查表；模式部分则直接结合 `src/modes.c`，给出 CTR 的计数器批量生成、GCM 的 CTR+GHASH 组合以及 XTS 的 tweak/XEX/CTS 流程。所有代码片段都由项目中的实际实现精简而来，而不是独立的演示伪代码。

整个实现始终保持三条不变量。第一，参考后端定义字节序、轮密钥次序和错误语义，优化后端只能改变内部数据表示，不能改变外部密文。第二，单块接口和多块接口共享同一 `sc_ctx`，批量宽度不足时只回退剩余分组，而不会重新扩展密钥。第三，工作模式只依赖统一分组接口，不直接识别 AESE、PSHUFB 或 GFNI；因此替换后端后，CTR/GCM/XTS 可以自动获得批量加密能力，同时仍由模式层负责计数器、认证和 ciphertext stealing 的正确性。

工程上把“功能完成”和“特定设备可执行”分开记录。源码可以为某条目标 ISA 生成正确指令，但只有运行时特征检测通过后才会进入该后端。这样既能保留 ARM64 与 x86-64 的同一套公共 API，也不会把静态编译成功误写成原生性能结果。本文所有性能结论均对应实际保存的 ARM 或 x86 测量样本。

安全边界：本项目是可复现实验实现，不是经过认证的生产密码库。尤其 T-table 会以秘密值索引缓存，不应成为真实系统的默认后端。

2 微架构基础与优化模型

2.1 吞吐率不是指令条数

现代处理器可多发射并行执行，实际耗时由执行端口、依赖链、指令时延和吞吐共同决定。因而同样的轮函数可以有三类改进：

1. 缩短关键依赖链，例如把互不相关的分组交错执行；
2. 减少内存层次开销，例如把状态保持在寄存器，或缩小表；
3. 用一条 SIMD/密码指令替代多条标量指令。

全展开可以删除循环控制，却会扩大指令缓存占用。本项目保留清晰的循环基线，只在数据表示和后端处做可测量的变换，同时以对象尺寸 CSV 记录代码代价。

分组密码的单个分组往往存在“第 $i+1$ 轮依赖第 i 轮”的长链，单纯增加 SIMD 指令并不能消除这条依赖。本项目因此把互不相关的四个或八个分组放入同一批次：当一个分组等待前一条指

令结果时，处理器仍可发射其他分组的轮指令。AES 硬件后端、SM4 shuffle/GFNI 和工作模式的计数器生成都采用这一思路。批量宽度是实现选择，不是 API 限制；调用者给出任意分组数，后端先处理完整批次，再用较窄内核或标量路径完成余数。

缓存行为同样影响结果。4 KiB T-table 可能全部驻留 L1，但仍会与轮密钥、输入输出和其他代码竞争；1 KiB 表减少数据占用，却增加旋转指令；固定扫描 shuffle 不按秘密值选择地址，但每轮需要读取或组合更多表行。报告因此同时记录吞吐、表大小和对象大小，不用“指令更少”替代实际测量。

2.2 端序与未定义行为

所有外部字节串都通过 `sc_load_be32/64` 与 `sc_store_be32/64` 显式转换。2 KiB SM4 重叠表虽然利用偏移读取旋转后的 32-bit 字，却不把非对齐地址强制转换成 `uint32_t*`，从而避免别名和对齐未定义行为。这一点由 ASan/UBSan 与 `-Werror` 共同约束。

AES 和 SM4 的规范都以字节串描述输入，但二者内部字排列不同；GIFT/TWINE 又以 bit 或 nibble 为基本单位。实现只在加载和保存边界进行显式转换，中间轮函数始终操作固定宽度无符号整数。移位量均小于类型宽度，旋转由左右移和按位或构成；所有可能非对齐的输入先复制到局部数组。因此原地操作、非对齐地址和不同编译器优化级别不会依赖宿主主机未规定的行为。

3 四种分组密码的基础实现

3.1 AES

AES 按 FIPS 197 实现 128/192/256-bit 密钥扩展、SubBytes、ShiftRows、MixColumns 及逆变换。参考路径以 16 字节状态表示；T-table 同时实现正向与逆向四表。ARM 路径用 AESE/AESMC 与 AESD/AESIMC，x86 路径用 AES-NI，多块时可进入 VAES。所有路径使用同一轮密钥并与 FIPS KAT 交叉比较。

输入按列映射为 4×4 字节状态。加密先异或第 0 轮密钥，中间轮依次执行 SubBytes、ShiftRows、MixColumns 和 AddRoundKey，末轮省略 MixColumns；AES-128/192/256 分别执行 10、12、14 轮。密钥扩展把原始密钥扩展为 $N_r + 1$ 组 128-bit 轮密钥，并根据 $N_k = 4, 6, 8$ 分别处理 RotWord、SubWord、Rcon 和 AES-256 的额外 SubWord 分支。

解密参考路径按相反顺序执行逆变换。为了让 AESD/AESIMC、AES-NI AESDEC 以及逆 T-table 都直接消费合适的轮密钥，初始化阶段同时准备逆序轮密钥，并对中间轮密钥执行 InvMixColumns。多块接口加载若干独立状态到寄存器，每轮对全部状态应用同一轮密钥；末尾不足并行宽度的分组交给窄批次处理。这样密钥扩展不进入稳态计时，也避免“硬件名称后端实际逐块调用参考代码”。

3.2 SM4

SM4 使用 32 轮广义 Feistel 更新：

$$X_{i+4} = X_i \oplus L(\tau(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)),$$

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24).$$

密钥扩展和数据轮均依据 GB/T 32907-2016。实现把输入、输出与轮密钥端序明确分开，并用官方向量 012345...3210 $\rightarrow$ 681edf...4246 验证。

数据路径只需维护四个 32-bit 滚动状态字。每轮先计算 $T = X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i$ ，再执行逐字节 S 盒 τ 和线性层 L ，最后与 X_i 异或。第 32 轮完成后以 $X_{35}, X_{34}, X_{33}, X_{32}$ 的逆序写出。滚动窗口减少了长期存活的临时量，批量后端则为每个 lane 保留独立四字状态，使多块之间没有数据依赖。

密钥扩展先将 128-bit 主密钥与系统参数 $FK_0 \dots FK_3$ 异或，再对 32 个 CK_i 常量执行与数据轮类似的非线性变换，但使用

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23).$$

产生的 32 个轮密钥按正序用于加密，按逆序用于解密，因此 SM4 的解密无需另写一套轮函数。T-table、shuffle、GFNI、AES 辅助和专用指令适配都遵守同一轮密钥次序，并在测试中同时比较加密和解密输出。

3.3 GIFT-64 与 TWINE

GIFT-64 采用四路 bitslice 状态，S 盒直接变为四个位平面上的布尔网络，位置换由固定掩码网络完成；批量 API 每次处理四个独立分组。TWINE 以 16 个 nibble 表示状态，分别实现 80-bit 与 128-bit 密钥调度；KAT 密文为 7c1f0f80b1df9c28 与 979ff9b379b5a9b8。

GIFT-64 的 64-bit 状态在 28 轮中依次经过 4-bit S 盒、固定 bit permutation 和轮密钥/轮常量异或。独立参考实现按 nibble 处理一个分组，用于确认轮常量、bit 编号和密钥更新；优化实现把四个分组的同一 bit 位置转置到四个位平面，再以 AND、XOR、NOT 表达 S 盒。固定置换被编译为掩码、移位和交换网络，解密使用逆布尔网络与逆置换，四个 lane 最后再转回字节序。

TWINE 的 64-bit 状态由 16 个 nibble 构成，共执行 36 轮。每轮只对偶数 nibble 执行“轮密钥异或、S 盒、异或到相邻 nibble”，随后执行固定置换；最后一轮不再置换。80-bit 与 128-bit 版本的区别主要在密钥状态宽度和轮密钥抽取/更新位置，数据轮可以共用。shuffle 后端把 16 个 nibble 打包进 128-bit 向量，第一次 TBL/PSHUFB 完成 S 盒查找，第二次重排完成轮置换，从而让多块数据布局和算法的 nibble 结构一致。

4 统一接口与运行时分派

公共上下文 `sc_ctx` 对外保持不透明，提供密钥初始化、单块/多块加解密、块长/算法/后端查询和明确错误码。单块 API 支持原地操作。后端可选 `ref`、三种 `ttable`、`shuffle`、`aes-hw`、`gfni`、`sm4-hw` 与 `auto`。

Listing 1: 统一接口的核心调用

```
sc_ctx ctx;
sc_status rc = sc_init(&ctx, SC_AES_128, SC_BACKEND_AUTO,
                      key, sizeof(key));
rc = sc_encrypt_blocks(&ctx, input, output, blocks);
```

ARM64 检测 AES、PMULL 与 SM4 能力；x86 检测 AES-NI、SSSE3、AVX2、VAES、PCLMUL、VPCLMUL 和 GFNI。显式请求不可用 ISA 时返回 `SC_ERR_UNSUPPORTED`，不做静默冒充。

`sc_init` 首先校验算法与密钥长度，再完成一次密钥扩展和后端选择。`auto` 按“可用的硬件/常量访问后端优先、参考后端兜底”的规则选择；显式后端则必须同时满足算法适用性和 CPU 特征。例如 AES 不能选择 GFNI，GIFT/TWINE 不能选择 T-table，缺少 VSM4 的处理器不能初始化 `sm4-hw`。最终实际选择可由 `sc_backend_id` 查询，便于测试和基准确认没有发生静默回退。

`sc_encrypt_blocks` 与 `sc_decrypt_blocks` 根据上下文直接进入批量内核。AES 硬件路径、SM4 shuffle/GFNI/AES 辅助路径、GIFT bitslice 和 TWINE shuffle 都在这里处理多块；只有尾端不足 lane 宽度时才调用较窄路径。API 允许输入输出地址相同，并在进入后端前检查上下文、指针和算法状态。CTR/GCM/XTS 因而只需按批次组织输入，不需要复制每种算法的密钥调度或轮函数。

5 优化技术

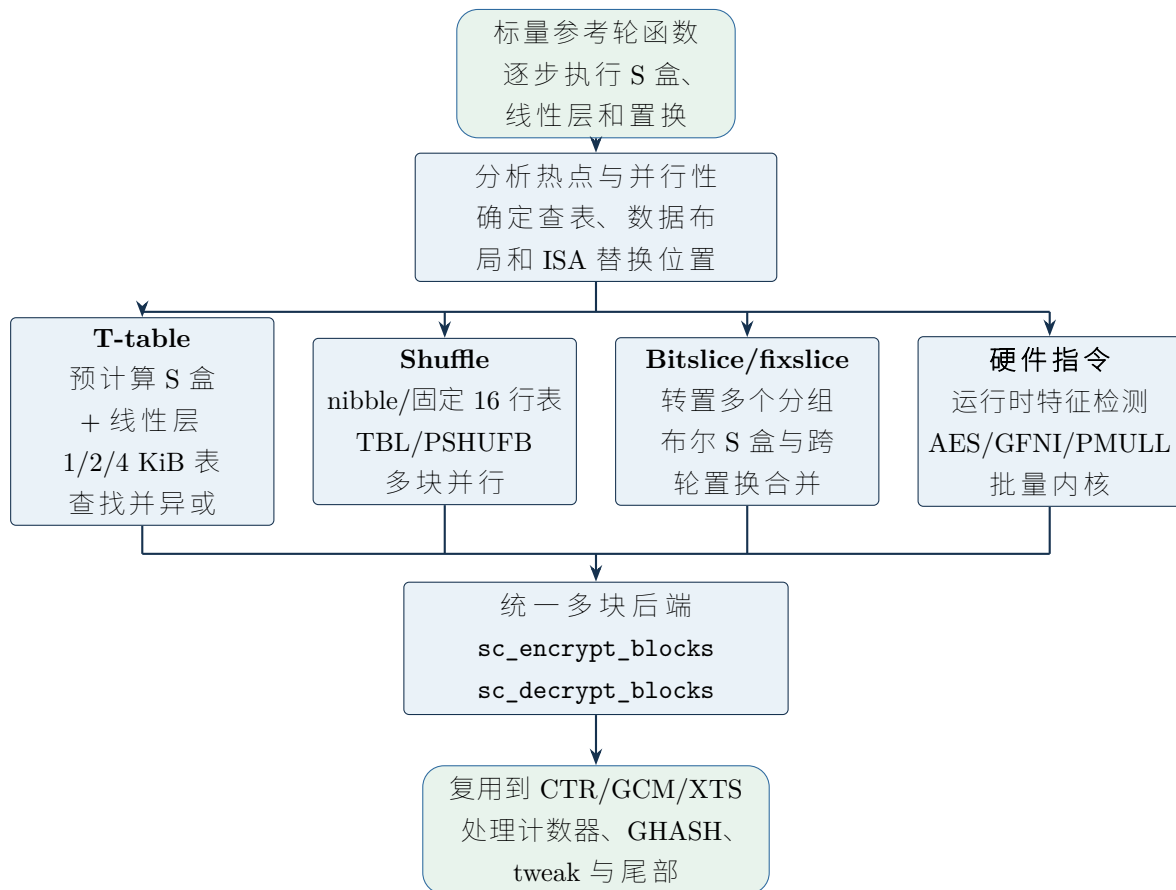


图 1: 从标量轮函数到 T-table、shuffle、bitslice/fixslice 和硬件指令后端的优化流程。

5.1 T-table: 空间换指令

T-table 的实现分两步。第一步在初始化时枚举全部 256 个字节，将 S 盒输出和线性变换预先合并；第二步在每一轮中从状态的四个字节分别查表并异或轮密钥。以 AES 为例，若 $s = S(i)$ ，则首表项包含 $\{02\}s, \{01\}s, \{01\}s, \{03\}s$ ，其余三张表由字节旋转得到。这样一次查表异或同时完成

SubBytes、ShiftRows 对应的取字节和 MixColumns。

设输入状态按四个大端字 s_0, s_1, s_2, s_3 保存，则一个输出字由

$$T_0[s_0^{(0)}] \oplus T_1[s_1^{(1)}] \oplus T_2[s_2^{(2)}] \oplus T_3[s_3^{(3)}] \oplus rk$$

得到。四个索引分别来自 ShiftRows 后落入同一列的字节，因此不需要先把 16 字节状态显式重排。每张表有 256 个 32-bit 项，共占 4096 字节；四个输出字可以交错计算，以减少单一查表结果形成的依赖链。表仅保存与密钥无关的 S 盒和线性变换，轮密钥仍在每轮末尾异或。

Listing 2: AES T-table 的构造与一轮轮更新 (src/aes.c)

```
for (unsigned i = 0; i < 256; ++i) {
    uint8_t s = aes_sbox[i];
    uint32_t x = ((uint32_t)gf_mul(s, 2) << 24) |
                ((uint32_t)s << 16) | ((uint32_t)s << 8) |
                gf_mul(s, 3);
    te[0][i] = x;
    te[1][i] = (x >> 8) | (x << 24);
    te[2][i] = (x >> 16) | (x << 16);
    te[3][i] = (x >> 24) | (x << 8);
}

t[0] = te[0][s0 >> 24] ^ te[1][(s1 >> 16) & 255] ^
        te[2][(s2 >> 8) & 255] ^ te[3][s3 & 255] ^ rk[0];
```

末轮不能包含 MixColumns，因此仍单独执行 S 盒、ShiftRows 和轮密钥异或；解密方向建立逆 S 盒与 InvMixColumns 合并的 `td[4][256]`，并对中间轮密钥应用逆列混合。SM4 的表项则预计算 $L(S(i) \ll 24)$ 。四表方案保存它的三个旋转版本；1 KiB 方案只保存首表并在轮内旋转；2 KiB 方案把每个 32-bit 表项连续复制两次，从偏移 3、2、1 读取旋转后的四字节序列。

SM4 的轮输入 T 被拆成 b_0, b_1, b_2, b_3 四个字节。4 KiB 方案直接计算 $T_0[b_0] \oplus T_1[b_1] \oplus T_2[b_2] \oplus T_3[b_3]$ ；1 KiB 方案从同一表取四个值后旋转 0、8、16、24 bit；2 KiB 方案利用每项连续复制的八个字节，让不同起始偏移读出的四字节窗口等价于旋转结果。三条路径的输出必须逐字等于标量 $L(\tau(T))$ ，解密只需把轮密钥反向后复用同一查表轮函数。

Listing 3: SM4 三种 T-table 数据路径 (src/sm4.c)

```
uint32_t a = sm4_t[0][b0], b = sm4_t[0][b1];
uint32_t c = sm4_t[0][b2], d = sm4_t[0][b3];

/* 1 KiB: one table plus rotations */
return a ^ ((b >> 8) | (b << 24)) ^
        ((c >> 16) | (c << 16)) ^ ((d >> 24) | (d << 8));

/* 2 KiB: duplicated entries permit overlapping byte loads */
return load32(overlap[b0]) ^ load32(overlap[b1] + 3) ^
        load32(overlap[b2] + 2) ^ load32(overlap[b3] + 1);
```

```
/* 4 KiB: four independent 256-entry tables */
return sm4_t[0][b0] ^ sm4_t[1][b1] ^
       sm4_t[2][b2] ^ sm4_t[3][b3];
```

表 1: 表布局与数据路径

算法/后端	表大小	每轮方法
AES 四表	4 KiB	SubBytes+ShiftRows+MixColumns 合并
SM4 四表	4 KiB	四个字节分别查表后异或
SM4 单表	1 KiB	查一个表，再旋转 8/16/24 bit
SM4 重叠	2 KiB	8 字节重复项，偏移 0/3/2/1 读取

表越小不保证越快：1 KiB 路径增加旋转，2 KiB 路径增加非对齐字节组装；4 KiB 路径则给 L1 数据缓存更大压力。四条路径同时存在，性能结论交给同一基准而不是先验判断。

T-table 的优化边界也很明确：它减少逻辑运算，却把由状态字节决定的索引暴露给缓存层次。攻击者如果能观察 cache set、预取或执行时间，可能推断密钥相关信息。因此运行时不会把“表路径最快”自动等价于“最安全”；报告把它作为一种数据路径单独测量，并与固定访问的 shuffle、bitslice 和硬件密码指令比较。

5.2 Shuffle、bitslice 与 fixslice

TWINE 的 4-bit S 盒天然适合 16-byte TBL：一条 TBL 并行完成 16 个 nibble 查找，第二次 TBL 完成轮置换。SM4 的 8-bit S 盒不能直接由单个 16 项表表示，本项目使用 16 行固定表和掩码选择，访问轨迹与秘密无关；这是常量时间实现路径，但单块时指令数高。GIFT 的布尔 S 盒和固定 bit permutation 完全不查秘密索引表，更接近 bitslice/fixslice 思路。

对 4-bit S 盒，输入 nibble 本身就是 16 项表的合法索引。ARM NEON TBL 和 x86 PSHUFB 都能在一个 128-bit 寄存器中同时完成 16 次查找。TWINE 后端先把多个分组的 nibble 排列为固定向量布局，执行 S 盒后用第二个常量索引向量完成轮置换；下一轮直接消费置换后的布局，避免每轮保存到内存再重新装载。解密使用逆 S 盒表和逆置换索引，接口语义与参考路径一致。

SM4 需要处理 8-bit S 盒。实现把输入拆成高、低 nibble，并固定遍历高 nibble 的 16 种可能值：每一行用低 nibble 做 TBL/PSHUFB 查找，再用“输入高 nibble 是否等于当前行号”的全字节掩码选择结果。循环次数、表行地址和加载顺序都与秘密无关。四个独立分组经过矩阵转置后同时执行 32 轮，最后再完成逆转置和大端写回；这样才能摊薄固定扫描 16 行造成的额外指令。

bitslice 则完全取消 S 盒表。GIFT 的四个输入分组先按 bit 位置转置为位平面，同一条 AND/XOR/NOT 同时作用于多个逻辑 S 盒；固定 bit permutation 通过掩码交换完成。fixslice 进一步让连续若干轮采用预先选择的数据布局，把一轮结束后的显式置换吸收到下一轮的 S 盒输入和轮密钥布局中。其代价是装入/写回时需要转置，因此小消息可能不占优势，而 CTR 等天然存在多个独立计数器的场景更容易填满并行 lane。

5.3 新指令集

- **AES:** ARM AESE/AESD 为本机四路运行路径；x86 AES-NI 与 VAES 已进入四路/八路实际后端并通过反汇编确认。ARM 的 AESE 完成 SubBytes、ShiftRows 和 AddRoundKey，随后 AESMC 完成列混合；x86 AESENC 把对应步骤合并为一条指令，VAES 则在更宽向量寄存器中同时处理多个 128-bit lane。解密分别使用 AESD/AESIMC 与 AESDEC/VAESDEC。
- **GHASH:** PMULL 做两个 64-bit 多项式的 Karatsuba 乘法，再以 $x^{128} = x^7 + x^2 + x + 1$ 两次折叠约减；x86 PCLMUL/VPCLMUL 已进入实际 GHASH 后端。输入在进入指令前转换为 GHASH 的多项式 bit 次序，三个部分积组合为 256-bit 中间值，约减后再转换回网络字节序。四块聚合使四个乘法彼此独立，给 PMULL/CLMUL 留出指令级并行空间。
- **GFNI:** SM4 S 盒按 $Y = A_1X + C_1$, $S(X) = A_2Y^{-1} + C$ 分解。标量 GFNI 模型穷举 256 个输入并逐项等于标准 S 盒，x86 源码用 affine、affine-inverse 和常量异或完成多个字节的 S 盒。矩阵和 bit 编号先由标量模型固定，再进入 SIMD 实现，避免“指令能执行但有限域基底不一致”的隐蔽错误。
- **AES 辅助 SM4:** 先用固定扫描预映射到 AES 逆 S 盒值，再以 AESE/AESENC 完成四块 SM4 S 盒并逆转 ShiftRows；全程不以秘密值直接索引缓存。该路径复用普及度更高的 AES 指令，在没有 SM4 专用指令时仍可对四个分组并行处理非线性层，线性层继续由旋转和异或完成。
- **专用 SM4 适配:** ARM SM4E/SM4EKEY 与 Intel VSM4RNDS4/VSM4KEY4 均已完成后端适配和静态指令检查。Apple M2 Pro 未报告 SM4E，Core i9-13900H 未报告 VSM4，因此没有专用 SM4 指令的原生性能数据。运行时显式请求该后端会返回 SC_ERR_UNSUPPORTED，而不会落到参考路径后继续标记为硬件。

表 2: 优化后端的数据宽度与进入条件

后端	典型批量宽度	主要指令/表示	进入条件与余数处理
AES ARM	4 块	AESE/AESD、AESMC/AESIMC	ARM AES feature 可用；不足四块由窄路径完成
AES x86	4/8 块	AES-NI、VAES	CPUID 与 XGETBV 同时允许相应向量状态
SM4 shuffle	4 块	TBL 或 PSHUFB 固定扫描	NEON/SSSE3 可用；尾块回退到较窄实现
SM4 GFNI	4 块	GF2P8AFFINE(INV)	x86 GFNI 可用且矩阵模型验证通过
GIFT/TWINE	4 块	bitslice 或 nibble shuffle	完整四块走批量内核，余数保持参考语义
GHASH	1/4 块	PMULL、PCLMUL、VPCLMUL	有 CLMUL 时执行多项式内核，否则用标量乘法

6 CTR、GCM 与 XTS

6.1 CTR

CTR 对四类密码全部开放。计数器按分组宽度大端递增，每批生成八个计数器并调用多块 API；末尾不足一块时只异或所需字节。调用前先验证全部计数器空间，回绕时不修改输出，并返回错误码 `SC_ERR_COUNTER_WRAP`。

具体流程是：先在临时副本上递增 $\lceil \text{len}/B \rceil$ 次完成回绕预检；随后每批复制并递增最多八个计数器，调用分组密码的批量加密生成密钥流，最后与输入异或。因此加密和解密使用同一函数，也能正确处理原地输入和不足整块的尾部。

这里的 B 由分组算法决定：AES/SM4 为 16 字节，GIFT-64/TWINE 为 8 字节；计数器宽度与分组宽度相同，均从最低有效的末字节向前按大端方式进位。预检使用计数器副本，只有确认整个请求不会回绕后才开始写输出和更新真实计数器，因此错误返回具有原子性。长度为零时不产生密钥流；尾部只使用最后一个密钥流块的前若干字节，但真实计数器仍按“已消费一个完整密钥流块”前进，避免下一次调用重复使用同一 counter block。

批量生成并不会改变 CTR 的安全条件：同一密钥下不得重复使用相同初始计数器。优化只把连续八个计数器一次交给分组密码，并没有改变计数器取值或异或顺序。由于密钥流先写入局部数组，输入与输出完全重合时仍会在覆盖输入字节前读取正确值。

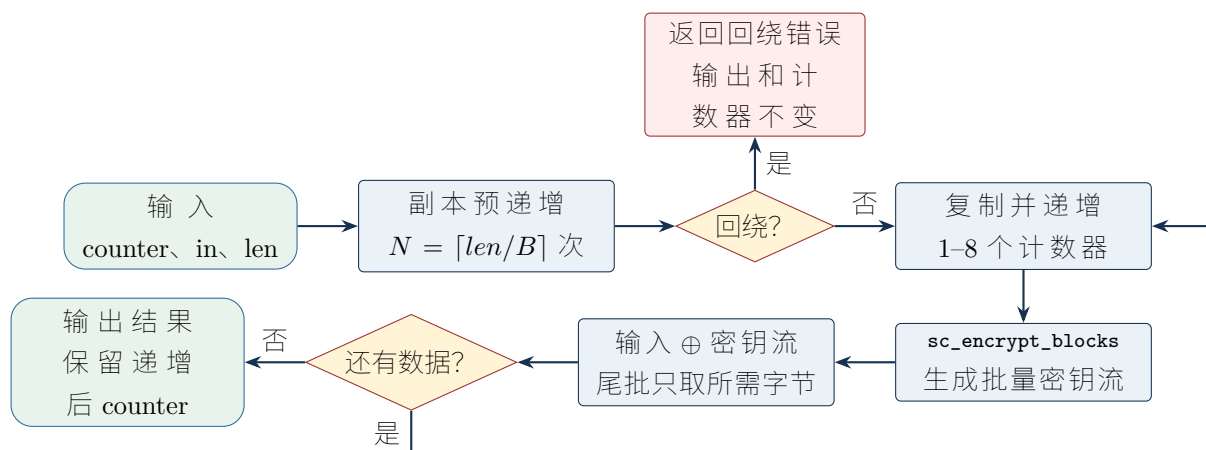


图 2: CTR 的回绕预检、八路密钥流生成与尾部处理。

Listing 4: CTR 的回绕预检和八路批量实现 (src/modes.c)

```

blocks = (len + block_size - 1) / block_size;
memcpy(probe, counter, block_size);
for (size_t i = 0; i < blocks; ++i)
    if (!increment_be(probe, block_size))
        return SC_ERR_COUNTER_WRAP; /* output is untouched */

while (off < len) {
    size_t left = (len - off + block_size - 1) / block_size;
    lanes = left < 8 ? left : 8;
    for (size_t lane = 0; lane < lanes; ++lane) {

```

```

    memcpy(counters + lane * block_size, counter, block_size);
    increment_be(counter, block_size);
}
sc_encrypt_blocks(ctx, counters, stream, lanes);
take = len - off < lanes * block_size
      ? len - off : lanes * block_size;
for (size_t i = 0; i < take; ++i)
    out[off + i] = in[off + i] ^ stream[i];
off += take;
}

```

6.2 GCM

GCM 只接受 128-bit 分组的 AES/SM4。96-bit IV 直接形成 $J_0 = IV \parallel 0^{31} \parallel 1$ ，其他 IV 先经 GHASH。AAD、密文和长度字段均按 SP 800-38D 处理。解密先以常量时间比较标签；失败返回 SC_ERR_AUTH 并清零输出。NIST AES-GCM 与 RFC 8998 SM4-GCM 向量均纳入回归。GHASH 预计算 H^2, H^3, H^4 ，完整的四块组用四个无数据依赖的乘法聚合。

实现首先计算 $H = E_K(0^{128})$ 。96-bit IV 直接组成 $J_0 = IV \parallel 0^{31} \parallel 1$ ，其他长度的 IV 也进入 GHASH。正文数据使用与 CTR 相同的八路批量加密，但只递增计数器低 32 位；认证部分依次吸收 AAD、密文和二者的 bit 长度，最后计算 $T = E_K(J_0) \oplus \text{GHASH}_H(A, C)$ 。四块聚合把四次串行更新 $Y_i = (Y_{i-1} \oplus X_i)H$ 改写为下式：

$$Y_4 = (Y_0 \oplus X_1)H^4 \oplus X_2H^3 \oplus X_3H^2 \oplus X_4H.$$

GHASH 在 $GF(2^{128})$ 上计算，约减多项式为 $x^{128} + x^7 + x^2 + x + 1$ 。AAD 和密文分别按 16 字节分块，最后不足一块在右侧补零；末尾再吸收两个大端 64-bit 长度字段，分别记录 AAD bit 数和密文 bit 数。非 96-bit IV 使用同一 GHASH 规则计算 J_0 ，并附加 IV 长度字段。这样快速路径只省略了 IV 的 GHASH，不改变后续 inc32、CTR 或标签计算。

加密流程先用 $J_0 + 1, J_0 + 2, \dots$ 的低 32 位计数器产生密文，再对 AAD 和密文认证。解密流程反过来先对收到的密文计算期望标签，并在 1...16 字节的请求标签长度上做无提前退出的异或归并比较；只有比较成功才执行 CTR 解密。标签错误、计数器回绕或后端失败都会清零明文输出，避免调用者误用未认证数据。计数器预检只检查低 32 位，因为 SP 800-38D 的 GCM counter block 使用 inc32，而不是对整个 128-bit J_0 进位。

四块聚合的关键并非减少乘法次数，而是把原来依赖前一块 Y_i 的四次乘法改写为可并行的四项。预计算 H^2, H^3, H^4 的成本在长消息上被摊薄；不足 64 字节或四块组后的尾部继续使用单块更新。因此标量、PMULL、PCLMUL 和 VPCLMUL 后端得到完全相同的 GHASH 值，只在乘法实现和并行宽度上不同。

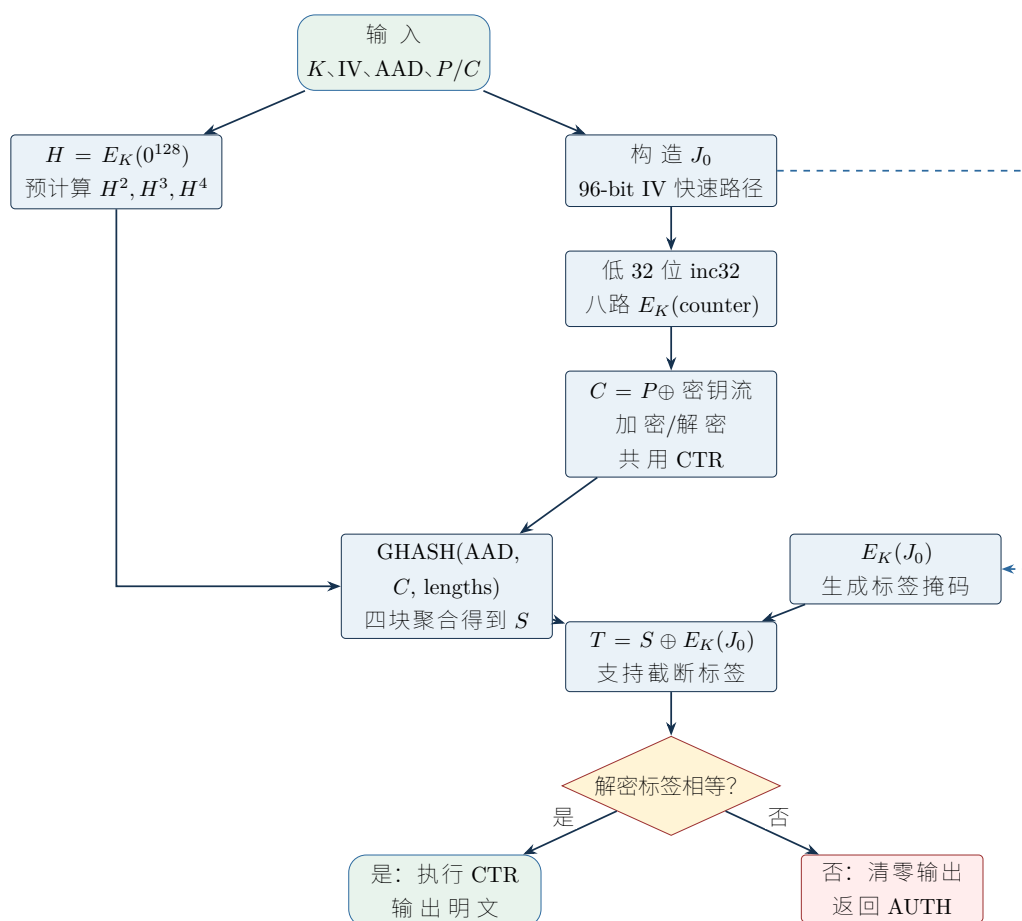


图 3: GCM 中 CTR 数据路径、四块 GHASH 与先认证后解密流程。

Listing 5: GCM 的四块 GHASH 聚合 (src/modes.c)

```

memcpy(h2, h, 16); ghash_mul(h2, h);
memcpy(h3, h2, 16); ghash_mul(h3, h);
memcpy(h4, h3, 16); ghash_mul(h4, h);

memcpy(term[0], y, 16);
xor16(term[0], data); /* Y0 xor X1 */
memcpy(term[1], data + 16, 16);
memcpy(term[2], data + 32, 16);
memcpy(term[3], data + 48, 16);
ghash_mul(term[0], h4);
ghash_mul(term[1], h3);
ghash_mul(term[2], h2);
ghash_mul(term[3], h);
memcpy(y, term[0], 16);
xor16(y, term[1]);
xor16(y, term[2]);
xor16(y, term[3]);

```

ghash_mul 在 ARM 上分派到 PMULL, 在 x86 上分派到 PCLMUL/VPCLMUL, 否则回退

到常量时间的逐 bit 标量乘法。解密在产生明文前先计算并常量时间比较标签；失败时清空输出。

6.3 XTS

XTS 使用两个独立密钥上下文，拒绝短于 16 字节的数据单元，并对非整块尾部执行 ciphertext stealing。IEEE 1619 约定在每个字节内左移并以 `0x87` 约减；GB/T 17964-2021 约定在每个字节内右移并以 `0xe1` 约减。OpenSSL 3.6 的 SM4-XTS 默认值为 GB，本项目同时保留 IEEE 选项。普通分组按八路预生成 tweak 并批量执行 XEX；CTS 在写回前保存尾部，因而支持原地加解密。

XTS 使用数据密钥 K_1 和 tweak 密钥 K_2 。先计算 $T_0 = E_{K_2}(i)$ ，再对第 j 个完整分组执行 $C_j = E_{K_1}(P_j \oplus T_j) \oplus T_j$ ，并令 $T_{j+1} = \alpha T_j$ 。实现一次生成八个连续 tweak，把前异或后的八个分组交给同一个批量后端，然后再异或各自 tweak。

公共函数 `sc_xts_tweak_from_data_unit` 把 64-bit 数据单元号编码成 16 字节初始输入：IEEE 约定写入低地址八字节并采用小端编码，GB/T 约定写入末八字节并采用大端编码。这个 16 字节值还不是每块直接使用的 tweak，必须先经过 E_{K_2} 得到 T_0 。初始化检查两个上下文的算法和分组长度完全一致，并拒绝 GIFT/TWINE 以及未知 tweak 枚举。

普通完整分组采用 XEX 结构：前异或把分组绑定到所在数据单元和块号，数据密钥完成真正的加/解密，后异或恢复字节表示。八路实现先在局部数组保存八个 tweak，再批量执行前异或、分组密码和后异或；因此解密可以直接选择 `sc_decrypt_blocks`，不需要逐块绕回公共单块接口。IEEE 与 GB/T 的差异只封装在 `xts_mul_x`，其余 XEX 和 CTS 控制流完全共用。

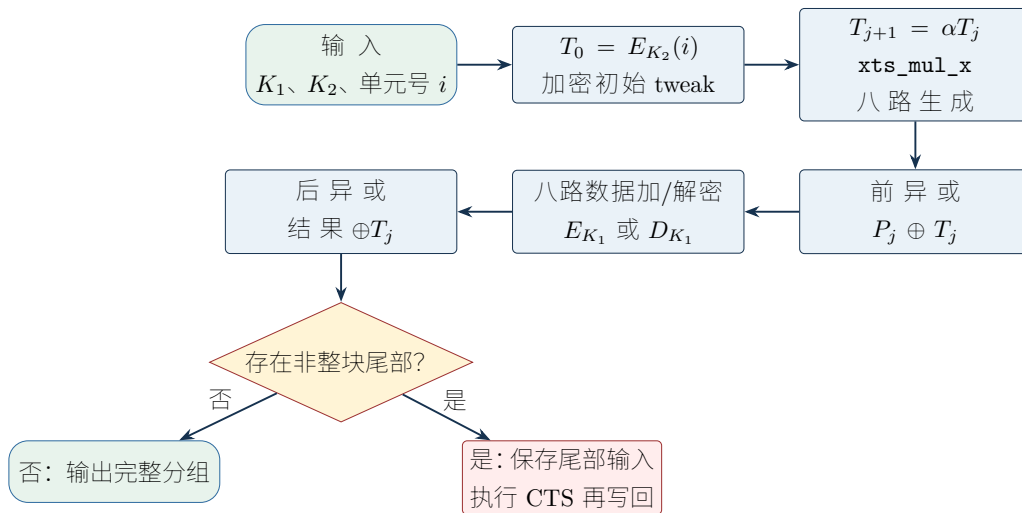


图 4: XTS 的双密钥、八路 tweak/XEX 与 CTS 分支。

Listing 6: XTS 的八路 tweak/XEX 流程 (src/modes.c)

```

uint8_t t[16];
sc_encrypt_block(tweak_ctx, tweak, t);
for (size_t lane = 0; lane < lanes; ++lane) {
    memcpy(tweaks + 16 * lane, t, 16);
    for (size_t i = 0; i < 16; ++i)
        work[16 * lane + i] = input[16 * (block + lane) + i]
            ^ t[i];
}

```

```

    xts_mul_x(t, convention);
}
sc_encrypt_blocks(data_ctx, work, work, lanes);
for (size_t lane = 0; lane < lanes; ++lane)
    for (size_t i = 0; i < 16; ++i)
        output[16 * (block + lane) + i] =
            work[16 * lane + i] ^ tweaks[16 * lane + i];

```

若数据单元含 m 字节尾部，CTS 先用当前 tweak 得到临时完整密文 CC，把 CC 的前 m 字节作为尾部密文，再用“尾部明文 || CC 剩余字节”和下一个 tweak 加密成倒数第一个完整密文。代码在覆盖输出前保存尾部输入，因此 17、31、63-byte 等原地加解密不会破坏尚未读取的数据。

解密 CTS 先用下一个 tweak 解开倒数第一个完整密文，得到由“尾部明文前缀”和 CC 后缀组成的临时块；再把收到的短尾部密文补到该后缀前方，使用当前 tweak 解出原倒数第一个完整明文。加密与解密都会在任何写回之前复制短尾部，解决输入输出相同且写入完整块会覆盖尾部的别名问题。整块长度时不进入 CTS；恰好 16 字节的数据单元只执行一次普通 XEX。

7 测试与工程验证

表 3: 验证层次

层次	内容
KAT	AES 三种密钥、SM4、GIFT-64、TWINE-80/128、NIST GCM、RFC 8998
内部交叉	参考/表/shuffle/硬件后端，固定种子随机输入，原地与非对齐语义
模式边界	CTR 尾部/回绕、GCM AAD/错误标签清零、XTS CTS/最小数据单元
OpenSSL	x86 上 20160 项 AES/SM4 ECB、CTR、GCM、XTS 随机差分；覆盖全部可用后端
工具链	-Wall -Wextra -Wpedantic -Werror、ASan、UBSan
ISA	ARM AESE/TBL/PMULL; x86 AES/VAES/PSHUFB/GFNI/CLMUL；SM4/VSM4 已适配，实验设备未报告支持

算法级验证。每种算法先运行标准 KAT，确保密钥扩展、端序和轮函数正确；随后用固定种子生成随机密钥与输入，把参考、T-table、shuffle 和所有可执行 ISA 后端逐块比较。AES 覆盖 128/192/256-bit 三种密钥，SM4 覆盖三种表布局以及 shuffle、AES 辅助、GFNI，GIFT/TWINE 同时比较单块参考与四块路径。测试不只验证密文，还用同一后端解密并检查恢复原文。

Listing 7: 后端与参考实现的批量交叉验证框架 (tests/test_symcrypto.c)

```

sc_init(&ref, algorithm, SC_BACKEND_REF, key, key_len);
sc_init(&opt, algorithm, requested_backend, key, key_len);

sc_encrypt_blocks(&ref, input, expected, blocks);
sc_encrypt_blocks(&opt, input, actual, blocks);

```

```
CHECK(memcmp(actual, expected, blocks * block_size) == 0);

sc_decrypt_blocks(&opt, actual, recovered, blocks);

CHECK(memcmp(recovered, input, blocks * block_size) == 0);
```

API 与边界验证。批量测试覆盖 0、1、并行宽度前后以及不能整除 lane 数的分组数量，并分别使用对齐、非对齐和原地缓冲区。CTR 检查四种算法的零长度、整块、尾部与全 0xff 附近回绕，确认失败时输入、输出和 counter 均不改变。GCM 覆盖空 AAD、任意长度 IV、截断标签、错误标签和失败清零。XTS 覆盖 16 字节最小单元、17/31/63 字节 CTS、512 B 与 4 KiB 数据单元、IEEE/GB 两种 tweak 约定以及原地加解密。

外部差分与 ISA 验证。OpenSSL 差分使用独立 API 计算 AES/SM4 的 ECB、CTR、GCM 和 XTS，包含非 96-bit GCM IV 与 XTS-CTS。x86 共完成 20160 项差分检查；运行时专用测试直接进入 AES-NI/VAES、PSHUFB、GFNI、PCLMUL/VPCLMUL 函数，而不是只测试自动分派。构建阶段再对实际后端符号反汇编，确认预期指令确实出现在相应函数中。ASan 检查越界和生命周期错误，UBSan 检查移位、对齐等未定义行为，警告则由 `-Werror` 提升为构建失败。

8 实验方法

ARM 实验平台为 Apple M2 Pro (Darwin arm64)，编译器为 clang-17.0.0 (clang-1700.0.13.5)；x86 实验平台为 13th Gen Intel(R) Core(TM) i9-13900H (Windows x86_64)，编译器为 clang-22.1.8 (MSYS2 UCRT64)，OpenSSL 为 OpenSSL 3.6.3 9 Jun 2026。消息长度取 64 B、512 B、4 KiB、8 KiB、64 KiB、1 MiB。每组预热 3 次、正式运行 15 次；每个样本至少处理 256 KiB，记录完整纳秒值，汇总 median、Q1、Q3、IQR、ns/byte 和十进制 GB/s。密钥扩展在计时区间外，模式的计数器、tweak、GHASH 和标签均在计时区间内。volatile checksum 防止编译器删除计算。

每个配置先完成密钥初始化，再执行 3 次不计入统计的预热，使代码页、表和数据进入稳定缓存状态。正式样本重复处理同一消息长度，内部循环次数按“总处理量至少 256 KiB”确定，避免 64 B 样本被计时器分辨率支配。每次循环都会把输出归入 checksum，保证编译器不能把已知输入的加密调用删除。分组基准分别记录加密和解密；模式基准把 counter/tweak 生成、异或、GHASH、标签和 CTS 都算入端到端时间。

15 个样本排序后取中位数作为中心值，取 Q1 与 Q3 的差 $IQR = Q3 - Q1$ 描述短时调度抖动。吞吐率按

$$\text{GB/s} = \frac{\text{处理字节数}}{\text{纳秒数}}$$

计算，使用十进制 GB；ns/byte 是其倒数形式，更适合观察低吞吐算法。报告保留全部样本，不只保存最终中位数：ARM 原始数据位于 `results/raw/native_samples.csv`，x86 原始数据位于 `results/raw/x86_samples.csv`，对应汇总分别位于 `results/summary/native_summary.json` 和 `results/summary/x86_summary.json`。

x86-64 结果同时记录单调时钟和序列化 RDTSCP。文中的 TSC ticks/byte 表示 invariant-TSC 计数，不等同于随睿频变化的物理核心周期。ARM 与 x86 结果分别保存，不把一个平台的数据改名为另一个平台。

两台机器的编译器、操作系统、缓存结构和频率策略不同，因此正文只在同一平台内计算后端

相对收益。跨平台图用于确认“某类优化是否同方向有效”，不把绝对 GB/s 差异直接归因于单条指令。x86 的 CPU 特征、测试计数和 ISA 执行状态另存于 `results/summary/x86_status.json`，用于区分“原生执行”与“仅完成目标 ISA 编译检查”。

9 结果与分析

9.1 ARM64 原生执行结果

ARM64 的 15 次正式测量样本保存在 `results/raw/native_samples.csv`，汇总统计保存在 `results/summary/native_summary.json`。本小节的图表和表格均使用这两份本机结果。

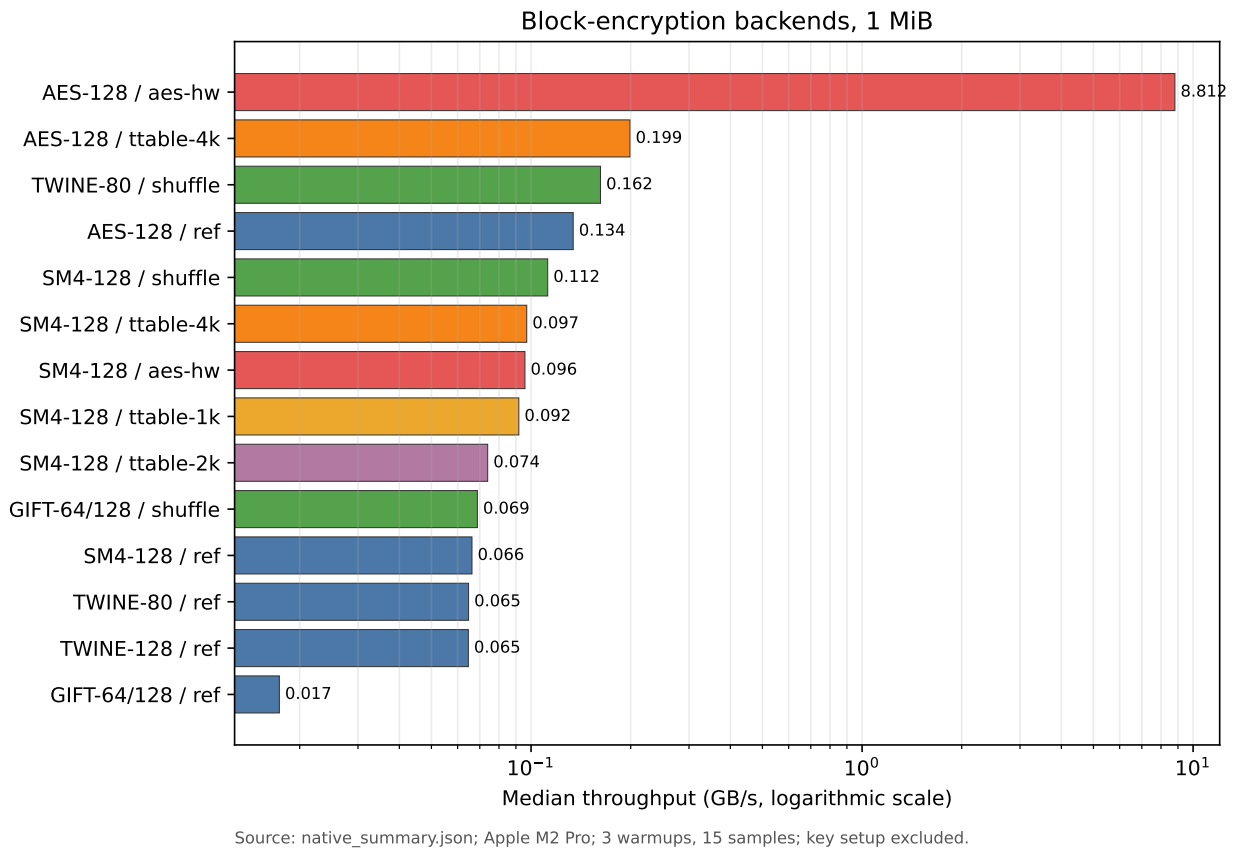


图 5: ARM64 上 1 MiB 分组加密吞吐率，横轴为对数尺度。

在 1 MiB 数据上，AES 参考、四表和硬件后端分别为 0.134、0.199、8.812 GB/s。T-table 相对参考为 1.48 $\times$ ，AESE/AESMC 为 65.72 $\times$ 。SM4 四表由 0.066 提升至 0.097 GB/s，即 1.46 $\times$ 。1 KiB 单表非常接近四表，2 KiB 重叠读取在本机反而较慢，说明“更小表”必须与额外旋转/装配成本一起评价。

硬件 AES 加密达到 8.812 GB/s，解密为 8.128 GB/s；两者都由四路 AESE/AESD 内核执行，解密稍低来自逆轮密钥和 AESIMC 数据路径。相比之下，AES 四表只有 0.199 GB/s，说明在支持专用指令的平台上，查表减少标量逻辑的收益远小于整轮硬件指令带来的收益。T-table 仍有价值，因为它揭示了没有密码扩展时“计算换内存”的性能边界。

SM4 的 ARM 结果更能区分不同软件方法：固定访问 shuffle 为 0.112 GB/s，相对 0.066 GB/s

参考路径约提升 1.70 倍；AES 指令辅助路径为 0.096 GB/s，与 4 KiB T-table 的 0.097 GB/s 接近。GIFT 四路 bitslice 从 0.017 提升至 0.069 GB/s，约 4.1 倍；TWINE-80 的 TBL 路径从 0.065 提升至 0.162 GB/s，约 2.5 倍。这说明轻量密码的主要收益来自多块数据布局，而不只是把一个标量 S 盒替换为向量查表。

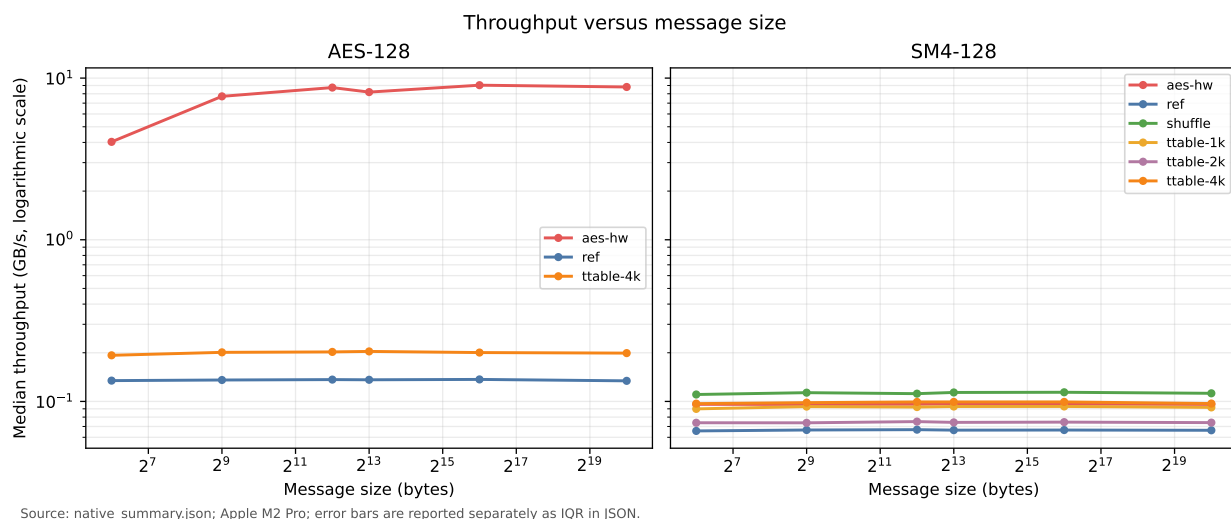


图 6: 不同消息长度下 AES/SM4 后端的稳态吞吐。

硬件 AES 在 64 B 时已经领先，随后趋于稳定；本实验把函数调用和循环调度算入短消息成本。SM4 shuffle 每轮固定扫描 16 行表，并以四块并行摊薄装配开销；其目标是展示无秘密相关访存，而不是声称单块最高性能。TWINE TBL 在完整批次上明显快于参考路径，但短消息仍会受到装入、nibble 重排和不足四 lane 的影响，说明向量查表必须与批量数据布局一起评价。

从长度曲线看，64 B 到 512 B 区间仍包含调用、分派和批次不满的固定开销；达到 4 KiB 后多数曲线趋于稳定。AES 硬件路径较早进入平台期，是因为轮函数本身很短，随后主要受加载、保存和循环调度限制。SM4 shuffle 需要完整四路数据才能摊薄转置，短消息上的相对收益因此低于 1 MiB 稳态结果。

AES-CTR 能直接发挥硬件块加密能力；XTS 还要执行第二个密钥的 tweak 加密与每块乘 α ，因此低于 CTR。GCM 已用 H^2-H^4 消除四块组内的串行依赖，跨组仍保留必要的 GHASH 依赖。IEEE 与 GB 两种 SM4-XTS 吞吐相近，符合两者只改变轻量 tweak 乘法的预期。

在 1 MiB 上，AES-CTR、GCM 加密和 XTS 加密分别为 1.727、0.913 和 1.429 GB/s。CTR 与 8.812 GB/s 原始分组内核之间的差距来自计数器生成、密钥流保存和逐字节异或；GCM 在此基础上还计算 GHASH，因此约为 CTR 的一半；XTS 需要两次 tweak 异或和每块乘 α ，但不含 GHASH，所以位于二者之间。GCM 解密为 0.880 GB/s，与加密接近，表明先认证后解密的额外读取没有改变主要瓶颈。

表 4: ARM64 上 1 MiB 全部实测结果

算法	后端	操作	GB/s	ns/byte	IQR
AES-128	ref	block	0.134	7.459	0.098
AES-128	ttable-4k	block	0.199	5.025	0.113

算法	后端	操作	GB/s	ns/byte	IQR
AES-128	aes-hw	block	8.812	0.113	0.009
AES-128	aes-hw	block-dec	8.128	0.123	0.004
SM4-128	ref	block	0.066	15.082	0.092
SM4-128	ttable-4k	block	0.097	10.305	0.174
SM4-128	ttable-1k	block	0.092	10.888	0.085
SM4-128	ttable-2k	block	0.074	13.522	0.156
SM4-128	shuffle	block	0.112	8.907	0.085
SM4-128	aes-hw	block	0.096	10.435	0.121
SM4-128	ttable-2k	block-dec	0.073	13.618	0.071
GIFT-64/128	ref	block	0.017	57.591	0.320
GIFT-64/128	shuffle	block	0.069	14.528	0.121
TWINE-80	ref	block	0.065	15.445	0.108
TWINE-80	shuffle	block	0.162	6.169	0.073
TWINE-128	ref	block	0.065	15.469	0.153
AES-128	aes-hw	ctr	1.727	0.579	0.016
SM4-128	ttable-2k	ctr	0.071	14.142	0.156
GIFT-64/128	ref	ctr	0.017	58.270	0.222
GIFT-64/128	shuffle	ctr	0.067	14.958	0.100
TWINE-80	shuffle	ctr	0.146	6.842	0.144
AES-128	aes-hw	gcm	0.913	1.095	0.029
AES-128	aes-hw	gcm-dec	0.880	1.136	0.038
SM4-128	ref	gcm	0.058	17.268	0.959
AES-128	aes-hw	xts	1.429	0.700	0.034
AES-128	aes-hw	xts-dec	1.387	0.721	0.026
SM4-128	ref	xts-ieee	0.061	16.310	0.204
SM4-128	ref	xts-gb	0.061	16.361	0.108
SM4-128	ref	xts-gb-dec	0.061	16.348	0.076

9.2 x86-64 原生执行结果

x86 状态文件记录为 **passed**。CPUID/XGETBV 检测到的 AES-NI、SSSE3、AVX2、VAES、GFNI、PCLMUL 与 VPCLMUL 均由专用测试直接调用，并与参考实现逐项比较。GFNI 硬件路径还穷举全部 256 个 SM4 S 盒输入；PCLMUL 和 VPCLMUL 则分别对照独立标量 GHASH oracle。Intel VSM4 后端已经完成适配和静态指令检查，但 Core i9-13900H 的 CPUID 未报告 VSM4 支持，因此显式请求 **sm4-hw** 返回 **SC_ERR_UNSUPPORTED**，本报告不提供 VSM4 原生吞吐量。

x86 的 15 次正式测量样本保存在 **results/raw/x86_samples.csv**，汇总统计保存在 **results/summary/x86_summary.json**，CPU 特征、测试状态与检查数量保存在 **results/summary/x86_status.json**。下述图表和表 4 使用这些 x86 实测文件。

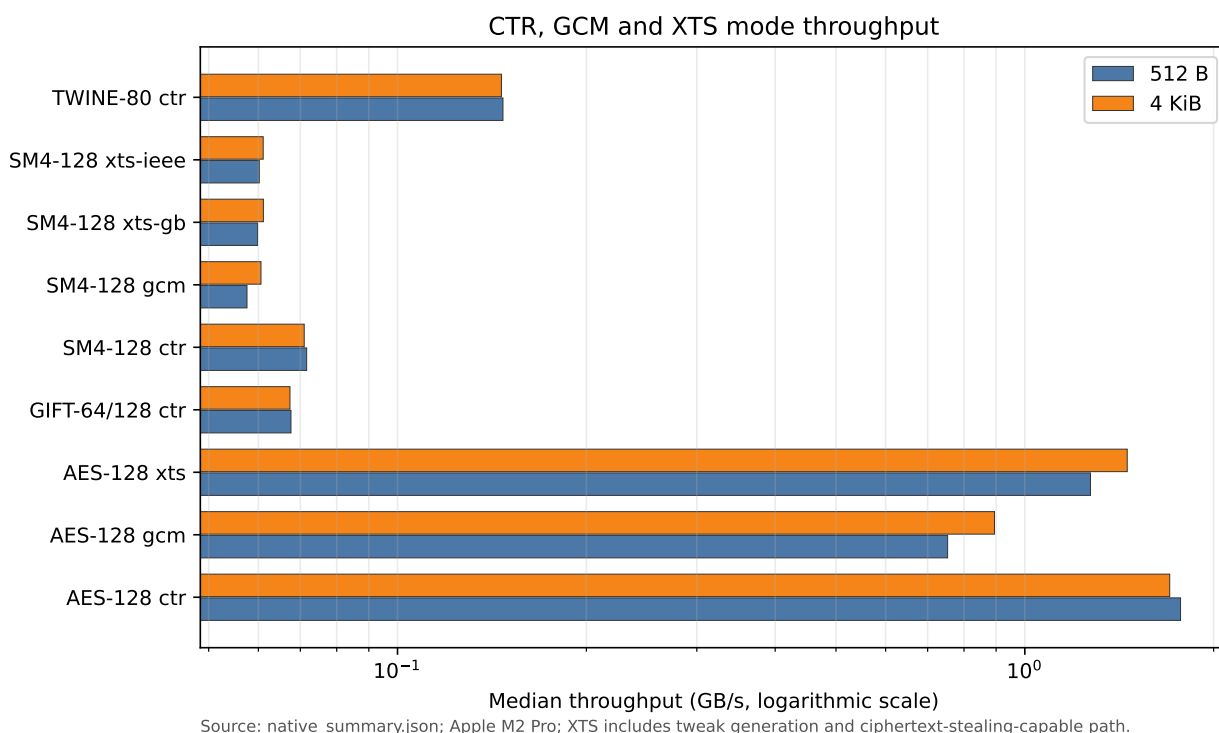


图 7: 512 B 与 4 KiB 的 CTR/GCM/XTS 吞吐率。

x86 的相对趋势与 ARM 一致，但硬件宽度和指令组合不同。AES 四表由 0.137 提升至 0.267 GB/s，约 1.95 倍；AES-NI/VAES 批量后端达到 21.313 GB/s，约为参考实现的 155.6 倍，解密为 16.157 GB/s。这里的硬件后端数字同时包含真实函数调用、循环和内存读写，不是单条 AESENC 的理论吞吐。

SM4 上，GFNI 为 0.299 GB/s，是 0.085 GB/s 参考路径的约 3.52 倍；PSHUFb shuffle 为 0.220 GB/s，约 2.59 倍；AES 辅助路径为 0.174 GB/s，约 2.05 倍。4 KiB、1 KiB、2 KiB T-table 分别为 0.132、0.127、0.093 GB/s，再次显示表越小并不自动越快。GFNI 加密和解密分别为 0.299 与 0.300 GB/s，符合 SM4 解密只反转轮密钥次序的结构。

GIFT 四路路径由 0.026 提升到 0.103 GB/s，接近 4 倍；TWINE-80 PSHUFb 路径由 0.065 提升到 0.346 GB/s，约 5.32 倍。后者高于 ARM 对应相对收益并不能单独归因于 PSHUFb，因为编译器、调度、缓存和主频均不同；可以确认的是，两台平台都证明了“向量查表必须配合多块布局”的方向性结论。

x86 的 1 MiB AES-CTR 为 2.129 GB/s，GCM 加/解密为 0.806/0.785 GB/s，XTS 加/解密为 1.230/1.248 GB/s。GCM 的比例下降说明 GHASH 和认证数据流成为主要开销，不能只凭 21.313 GB/s 的 AES 分组速度推测模式吞吐。SM4 AES 辅助后端在 GCM 中达到约 0.147 GB/s、在 GB-XTS 中达到约 0.159 GB/s，均明显高于参考模式；模式层批量调用确实把分组后端收益传递到了端到端实现。

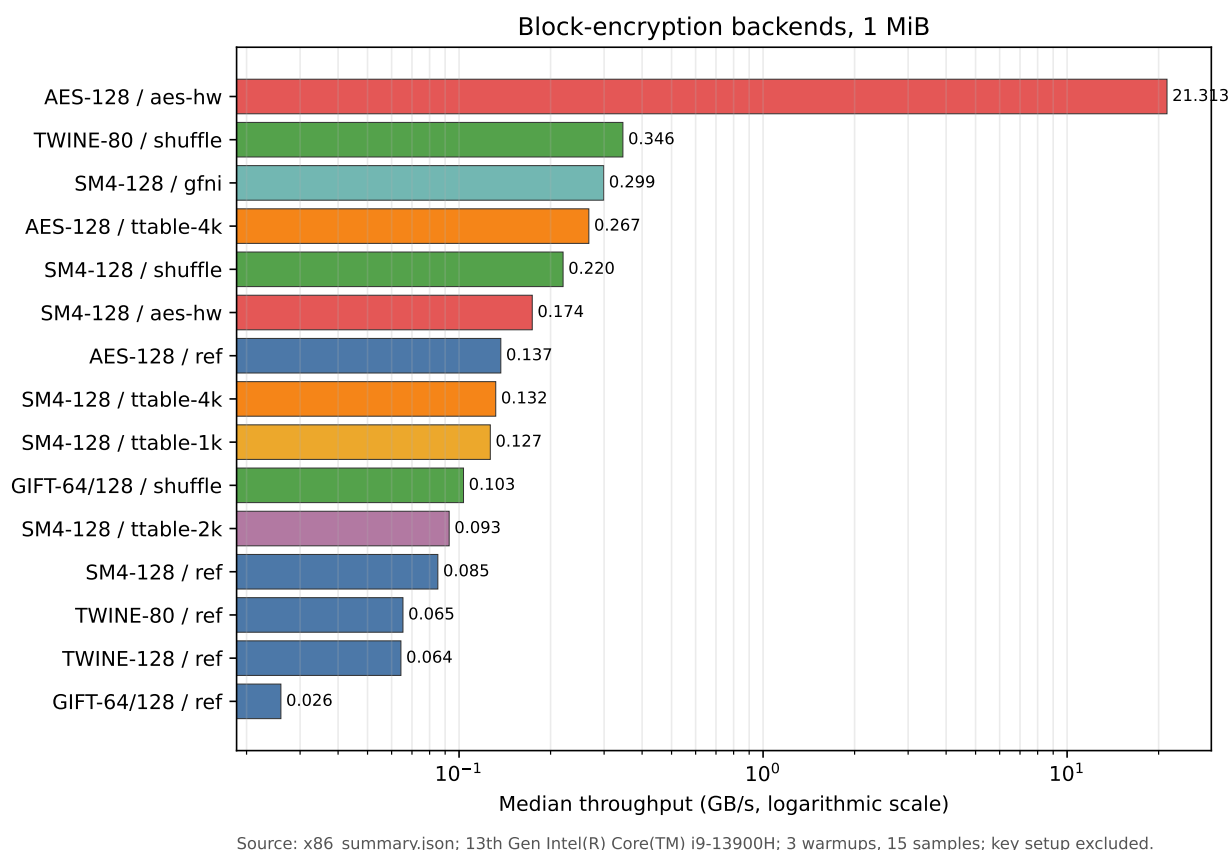


图 8: x86-64 上 1 MiB 分组加密吞吐率; 数据来自独立 x86 汇总。

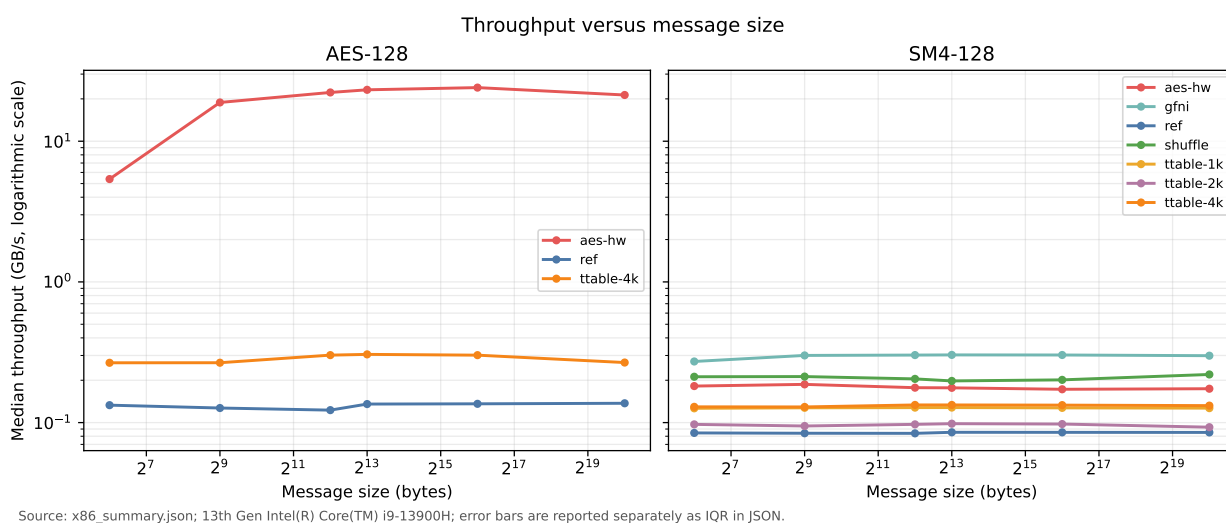


图 9: x86-64 上 AES/SM4 后端随消息长度的吞吐变化。

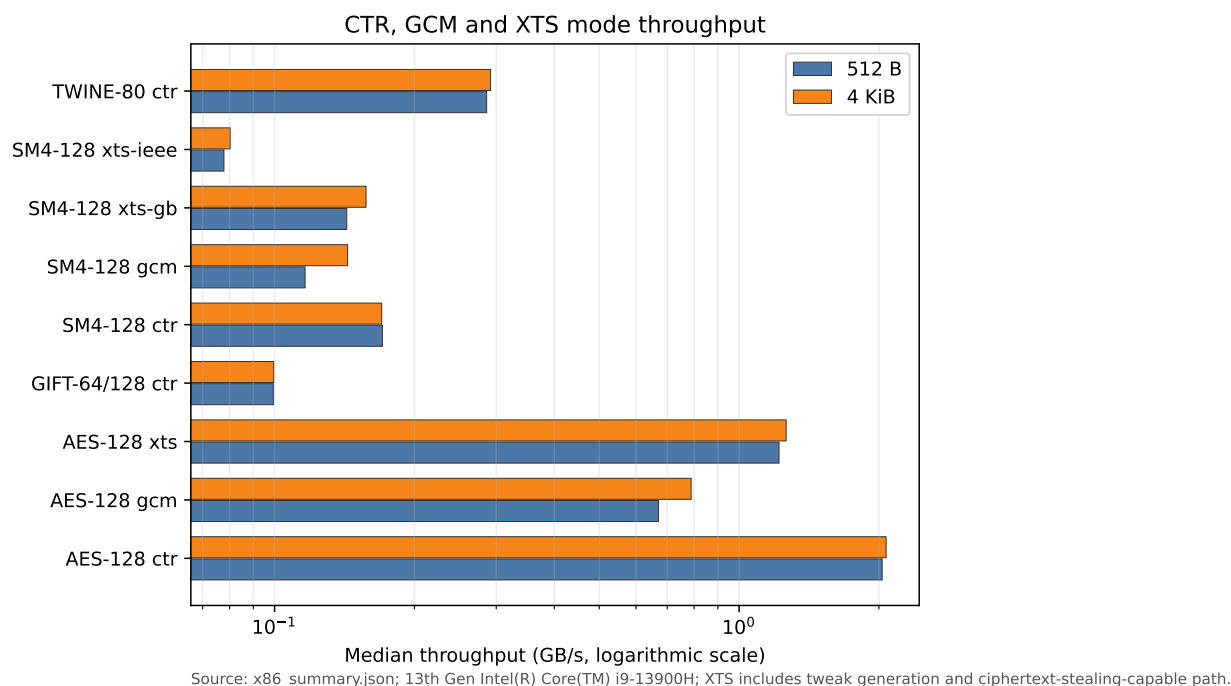


图 10: x86-64 上 CTR、GCM 与 XTS 的 512 B/4 KiB 吞吐率。

表 5: x86-64 上 1 MiB 全部实测结果

算法	后端	操作	GB/s	ns/byte	TSC ticks/byte
AES-128	ref	block	0.137	7.289	21.832
AES-128	ttable-4k	block	0.267	3.742	11.208
AES-128	aes-hw	block	21.313	0.047	0.141
AES-128	aes-hw	block-dec	16.157	0.062	0.186
SM4-128	ref	block	0.085	11.744	35.182
SM4-128	ttable-4k	block	0.132	7.574	22.686
SM4-128	ttable-1k	block	0.127	7.893	23.641
SM4-128	ttable-2k	block	0.093	10.779	32.288
SM4-128	shuffle	block	0.220	4.548	13.624
SM4-128	aes-hw	block	0.174	5.744	17.206
SM4-128	gfni	block	0.299	3.345	10.020
SM4-128	ttable-2k	block-dec	0.096	10.435	31.255
SM4-128	gfni	block-dec	0.300	3.329	9.978
GIFT-64/128	ref	block	0.026	38.550	115.467
GIFT-64/128	shuffle	block	0.103	9.669	28.960
TWINE-80	ref	block	0.065	15.295	45.812
TWINE-80	shuffle	block	0.346	2.891	8.660
TWINE-128	ref	block	0.064	15.533	46.525
AES-128	aes-hw	ctr	2.129	0.470	1.407

算法	后端	操作	GB/s	ns/byte	TSC ticks/byte
SM4-128	ttable-2k	ctr	0.090	11.082	33.192
SM4-128	aes-hw	ctr	0.170	5.886	17.629
GIFT-64/128	ref	ctr	0.026	38.876	116.443
GIFT-64/128	shuffle	ctr	0.099	10.089	30.219
TWINE-80	shuffle	ctr	0.294	3.406	10.202
AES-128	aes-hw	gcm	0.806	1.240	3.715
AES-128	aes-hw	gcm-dec	0.785	1.274	3.816
SM4-128	ref	gcm	0.076	13.194	39.528
SM4-128	aes-hw	gcm	0.147	6.815	20.413
SM4-128	aes-hw	gcm-dec	0.148	6.741	20.191
AES-128	aes-hw	xts	1.230	0.813	2.434
AES-128	aes-hw	xts-dec	1.248	0.801	2.401
SM4-128	ref	xts-ieee	0.080	12.577	37.671
SM4-128	ref	xts-gb	0.080	12.544	37.573
SM4-128	ref	xts-gb-dec	0.093	10.720	32.109
SM4-128	aes-hw	xts-gb	0.159	6.309	18.897
SM4-128	aes-hw	xts-gb-dec	0.156	6.416	19.218

9.3 ARM64 与 x86-64 对照分析

总体趋势。把两组原生结果放在一起看，可以得到“基础软件路径接近、x86 专用向量后端优势更大、模式吞吐不与分组吞吐成比例”的总体结论。两组数据来自不同 CPU、操作系统和编译器，因此下面的比例用于描述本项目的两套实测实现，而不是严格受控的微架构优劣结论。

AES 对比。AES 标量参考在 ARM64 与 x86-64 上分别为 0.134 和 0.137 GB/s，几乎处于同一水平；四表路径分别为 0.199 和 0.267 GB/s，x86 约高 34%。进入硬件路径后差距明显扩大：ARM AESE/AESMC 为 8.812 GB/s，相对本平台参考实现提升 65.72 倍；x86 AES-NI/VAES 为 21.313 GB/s，相对参考实现提升约 155.6 倍，绝对吞吐约为 ARM 的 2.42 倍。

SM4 与轻量密码对比。SM4 的 ARM/x86 参考实现分别为 0.066/0.085 GB/s，4 KiB T-table 为 0.097/0.132 GB/s，shuffle 为 0.112/0.220 GB/s，AES 辅助路径为 0.096/0.174 GB/s。x86 的优势从参考路径约 1.29 倍扩大到 shuffle 的约 1.96 倍，并且还能使用 ARM 平台没有的 GFNI 路径达到 0.299 GB/s。轻量密码方面，GIFT 批量路径在 ARM/x86 上为 0.069/0.103 GB/s，TWINE-80 为 0.162/0.346 GB/s。两台平台都证明 bitslice 或 shuffle 必须与多块布局结合，而 x86 在 TWINE 的 PSHUFB 路径上收益更明显。

工作模式对比。模式吞吐没有简单复制原始 AES 的 2.42 倍差距：AES-CTR 在 ARM/x86 上为 1.727/2.129 GB/s，x86 约高 23%；AES-GCM 为 0.913/0.806 GB/s，ARM 反而约高 13%；AES-XTS 为 1.429/1.230 GB/s，ARM 约高 16%。这说明 GCM 的 GHASH、XTS 的 tweak/XEX、计数器生成和内存调度会重新决定端到端瓶颈，因此模式性能不能由分组峰值直接推算。共同结论是：硬件 AES 优势明显；固定访问/bitslice 依赖多块；模式还需优化 GHASH 等外围路径。

10 安全、性能与平台权衡

1. **T-table**: 通常减少指令, 但秘密相关索引会暴露缓存访问模式; 选择该后端时必须同时评估吞吐率与侧信道风险。
2. **Shuffle/bitslice**: 访问模式固定, 更适合常量时间; 需要足够多并行块才能摊薄转置和数据装配。
3. **硬件密码指令**: AES/PMULL 兼顾性能和固定执行路径, 但仍不能替代协议级 nonce 管理、密钥生命周期和故障防护。
4. **运行时分派**: 可移植二进制必须检查 CPUID/XGETBV 或 ARM feature; “编译器接受 intrinsic” 不等于当前 CPU 可以执行。
5. **x86 平台**: AES-NI/VAES、SSSE3、GFNI 与 CLMUL 已完成原生执行和性能采样; VSM4 已完成适配, 但实验设备不支持该指令集。

表 6: 不同实现路径的性能来源与安全关注点

路径	性能来源	需要关注的边界
标量参考	数据表示直接、便于审计与交叉验证	指令数多; 若直接使用普通 S 盒表仍可能有秘密相关访问
T-table	合并 S 盒、线性层和置换, 减少轮内指令	索引依赖秘密, 缓存/预取轨迹可能泄漏; 表尺寸影响 L1
Shuffle	固定表地址, 一条指令并行多个 nibble/byte	需要扫描、转置和足够多 lane; 固定访问不等于完整侧信道认证
Bitslice/fixslice	用布尔网络并行多个分组, 取消秘密索引	装入/写回转置成本高, 密钥调度与轮布局更复杂
硬件密码指令	一条指令完成整轮或有限域乘法, 依赖链更短	必须正确检测 CPU/OS 状态; 仍需保护密钥、nonce 和错误路径

工作模式还引入了与轮函数无关的安全条件。CTR 和 GCM 在同一密钥下不得重复 counter/IV, 否则密钥流复用会直接泄漏明文关系; GCM 标签长度越短, 伪造成功概率越高, 调用者还必须限制失败尝试次数。XTS 的 tweak 不是 nonce, 但数据单元号必须稳定且与扇区位置一致; XTS 只提供存储加密的窄块语义, 不提供消息完整性。项目实现回绕检测、标签常量时间比较和失败清零, 是为了保证 API 边界正确, 但这些检查不能替代上层协议的密钥轮换和状态管理。

“固定访问” 也不等于已经通过侧信道认证。shuffle/bitslice 消除了最明显的秘密索引表, 但实际系统仍可能受到分支预测、编译器重写、共享执行资源、功耗、电磁和故障注入影响。硬件 AES/GFNI/CLMUL 通常提供更稳定的数据路径, 仍需结合目标处理器文档和部署威胁模型评估。因而本报告把性能、访存模式和平台可用性分开陈述, 不给任何后端附加超出测试证据的安全等级。

11 结论

本项目建立了从正确性基线到多类优化和三种工作模式的完整链条。实测说明：表优化的收益受缓存与额外指令共同约束；硬件 AES 的数量级优势最稳定；shuffle/fixslice 必须和多块数据布局一起设计；GCM/XTS 不能只优化分组密码而忽略 GHASH、计数器和 tweak。ARM64 与 x86-64 均已完成原生验证；x86 结果额外证明 GFNI、VAES、PCLMUL/VPCLMUL 的实际执行，并以独立文件保存 RDTSCP TSC ticks/byte。VSM4 后端已经完成适配和静态指令检查，但实验使用的 Core i9-13900H 不支持该指令集，因此没有 VSM4 原生性能数据。

在算法层，AES/SM4 的参考、T-table、shuffle 和硬件路径使用同一密钥语义，GIFT/TWINE 则证明 bitslice 与 nibble shuffle 可以把轻量密码扩展到多块并行。T-table 展示了空间换计算的经典方法，但 1/2/4 KiB 方案的实测差异说明缓存占用、旋转和非对齐装配必须同时考虑。ARM AESE/AESD、x86 AES-NI/VAES、GFNI 以及 PMULL/PCLMUL/VPCLMUL 的原生结果进一步说明，专用指令的收益只有在批量内核、正确轮密钥布局 and 真实运行时分派共同成立时才能转化为端到端吞吐。

在模式层，CTR 通过八路计数器直接复用多块加密；GCM 把八路 CTR 与四块 GHASH 聚合组合，并保持任意 IV、AAD、截断标签和先认证后解密语义；XTS 把八路 tweak/XEX、IEEE/GB 两种乘法约定和原地安全 CTS 统一到双上下文接口。结果证明，最快的分组密码后端并不自动得到同等比例的 GCM/XTS 提升，因为 GHASH、异或、计数器、tweak 和尾部控制流会成为新的瓶颈。

最终实现同时保留 ARM64 与 x86-64 的原始样本、汇总统计、平台特征和指令检查证据。可执行的后端已经完成 KAT、随机交叉、OpenSSL 差分、sanitizer 和原生性能测试；已适配但设备不支持的 SM4E/VSM4 则明确停留在编译与静态指令检查层级。由此，报告中的每项功能、优化方法和性能结论都能对应到具体代码路径与相应验证证据。

参考文献

- [1] NIST, *FIPS 197: Advanced Encryption Standard*, 2023. <https://csrc.nist.gov/pubs/fips/197/final>
- [2] 国家标准化管理委员会, *GB/T 32907-2016 SM4 分组密码算法*. <https://openstd.samr.gov.cn/bzgk/std/newGbInfo?hcno=7803DE42D3BC5E80B0C3E5D8E873D56A>
- [3] Banik et al., *GIFT: A Small Present*, CHES 2017. <https://www.iacr.org/archive/ches2017/10529227/10529227.pdf>
- [4] Suzaki et al., *TWINE: A Lightweight Block Cipher for Multiple Platforms*, SAC 2012. https://jpn.nec.com/rd/tg/dss/pdf/twine_SAC_full_v5.pdf
- [5] NIST, *Block Cipher Modes*. <https://csrc.nist.gov/projects/block-cipher-techniques/bcm>
- [6] P. Yang, *RFC 8998: ShangMi Cipher Suites for TLS 1.3*, 2021. <https://www.rfc-editor.org/rfc/rfc8998.html>
- [7] W. Guo, *Efficient Constant-Time Implementation of SM4*, 2022/1154. <https://eprint.iacr.org/2022/1154.pdf>
- [8] OpenSSL, *EVP_EncryptInit: xts_standard*. https://docs.openssl.org/3.4/man3/EVP_EncryptInit/